IN432 | M1

LEBANESE UNIVERSITY

Database Integration with Flask

Session 4

M1 Computer Science

Lina’s Task API: from Python objects to persistent, related data.

Our Starting Point: A Working Task API

RECALL THE PREVIOUS LAB

Request	Purpose
POST /api/tasks	Create a task
GET /api/tasks	List tasks
GET /api/tasks/{id}	Read one task
PATCH /api/tasks/{id}	Update selected fields
DELETE /api/tasks/{id}	Delete a task

Lina creates a task. How will the API keep it after a restart?

THEORY | EXPLAIN BEFORE IMPLEMENTING

Separate HTTP Logic from Database Logic

THEORY | RESPONSIBILITIES

Responsibility	Location	Job
HTTP route	Existing Blueprint module	Receive requests; return responses
Validation	Existing validation code	Check fields and values
Store	task_api/store.py	Read and change stored data
Model	task_api/models.py	Describe tables and mapped objects
Extension	task_api/extensions.py	Create the shared db object
Configuration	task_api/__init__.py	Configure Flask; initialize db

Request → route → validation → store → model / database.

What Is SQLAlchemy? Is an ORM (Object Relational Mapper) Required?

Direct driver

PYTHON sqlite3

- Write SQL yourself.
- Bind values and read rows.
- Manage transactions.

SQLAlchemy Core

SQL TOOLKIT

- Build SQL expressions.
- Work with tables and rows.
- Use an Engine / connection.

SQLAlchemy ORM

OUR APPROACH

- Map Python classes to tables.
- Read and modify objects.
- Use an ORM Session.

SQLAlchemy is a Python database toolkit. Flask does not require an ORM.

Understand the Names Before Using Them

Flask-SQLAlchemy

THE INTEGRATION

- A package imported as flask_sqlalchemy.
- SQLAlchemy is its extension class.
- db is the instance we create.

What db exposes

TWO DIFFERENT ROLES

- db.Model: the mapped-model base class.
- db.session: tracks database work.
- db.init_app(app): connects configuration.

db is our extension object; it is not a database file or a single connection.

Apply It: Create db and Configure the App

Two existing files | selected lines

```
# task_api/extensions.py
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()

# task_api/__init__.py: inside create_app()
# db is imported from .extensions
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///tasks.db"
db.init_app(app)
```

Create the extension once; initialize it after setting the database URI.

This relative SQLite file is stored in Flask's instance directory.

What Is an ORM Model?

THEORY | CLASS, OBJECT, AND ROW

- **A model is a Python class** mapped to a database table.
- **An instance represents** application data for one potential or existing row.
- **Mapped attributes connect** object values to table columns.
- **The model describes storage.** Defining it does not create a table or insert a row.

Class → mapping. Object → values. Database row → persisted data.

Next: read the declarations, then define the Task model.

How to Read a Model Declaration

THEORY | INHERITANCE AND COLUMN METADATA

Declaration	Meaning
class Task(db.Model)	Inherit the mapped-model base; like Java extends
__tablename__ = "tasks"	Choose the database table name
db.Column(type, ...)	Declare a mapped column on the class
primary_key=True	Identify each row
nullable=False	Reject SQL NULL; it does not reject empty text
default=False	ORM insertion default when no value is supplied

The inherited constructor accepts mapped names: Task(title=..., done=False).

Apply It: Define the Task Model

task_api/models.py | mapping excerpt

```python
from .extensions import db

class Task(db.Model):
    __tablename__ = "tasks"
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(120), nullable=False)
    done = db.Column(db.Boolean, nullable=False, default=False)
```

Task inherits mapping behavior; id, title, and done map to columns.

This is a mapping excerpt; the starter also declares constraints.

Why Convert a Model Object Before Returning It?

THEORY | PERSISTENCE MODEL AND API REPRESENTATION

- **A Task object has ORM** behavior; it is not a JSON response.
- **Serialization selects** the values the API is allowed to expose.
- **A Python dictionary is** an intermediate representation that Flask can encode as JSON.
- **We write to_dict() ourselves.** It reads fields; it does not save the task.

Database row → Task object → public dictionary → JSON response.

Next: choose the public fields explicitly in Task.to_dict().

Apply It: Define the Public Task Representation

task_api/models.py | method inside Task

```
def to_dict(self):
    return {
        "id": self.id,
        "title": self.title,
        "done": self.done
    }
```

self refers to this Task instance, like Java this.

The route encodes the returned dictionary as JSON.

When Does Flask Know Which App Is Active?

THEORY | APPLICATION CONTEXT AND TABLE CREATION

Execution situation	Application context
HTTP request	Available automatically
Registered Flask CLI command	Available automatically
Independent Python script	Create one explicitly

An application context makes the current Flask app, its configuration,
and its scoped Session available.
create_all() creates missing tables; it does not update an existing schema.

Apply It: Run Database Code Without an HTTP Request

initialize_db.py | illustrative new file at the project root

```
from task_api import create_app
from task_api.extensions import db
from task_api.models import Task

app = create_app()
with app.app_context():
    db.create_all()
    print("Tables are available.")
```

Run from the project root: python initialize_db.py.

The with block supplies the context; no web server is required.

What Is db.session?

Tracks objects

UNIT OF WORK

Registers new objects for insertion.

Detects changes to loaded objects.

Marks objects for deletion.

Coordinates a transaction

DATABASE WORK

The Engine manages connections.

Flushes pending changes as SQL.

Commits or rolls back the transaction.

This ORM Session is scoped to the Flask app context; it is not a login session.

THEORY | EXPLAIN BEFORE IMPLEMENTING

Session Operations: What Does Each One Do?

THEORY | KEEP THIS REFERENCE NEAR THE CODE

Operation	Purpose
add(object)	Track a new object for insertion
add_all(objects)	Track several new objects
get(Model, id)	Find by primary key; may reuse a loaded object
execute(statement)	Execute a statement
flush()	Send pending changes into the transaction
commit()	Flush and commit the transaction
rollback()	Roll back the current transaction
delete(object)	Mark an object for deletion
refresh(object)	Reload attributes from the database

execute() runs a statement; flush() sends changes; commit() completes the transaction.

Apply It: Follow a New Task Through the Session

Demonstration | inside an active app.app_context() block

```
task = Task(title="Prepare project proposal", done=False)
print(task.id)              # None: only a Python object

db.session.add(task)        # register for insertion
db.session.flush()          # INSERT inside the transaction
print(task.id)              # generated primary key

db.session.commit()         # commit the transaction
```

An object can have a generated ID before its transaction is committed.

Here flush() is explicit for teaching; commit() would also flush.

Quick Check: Is This Task Safely Stored Yet?

Demonstration code | inside an active application context

```
task = Task(title="Prepare project proposal", done=False)

db.session.add(task)
db.session.flush()

print(task.id)
```

Pair discussion: what happens if we roll back immediately after this code?

Predict both the printed ID and the final database row.

EXPLANATION | CONNECT THE CODE TO THE THEORY

Answer: Flush Sends SQL; Commit Finishes the Transaction

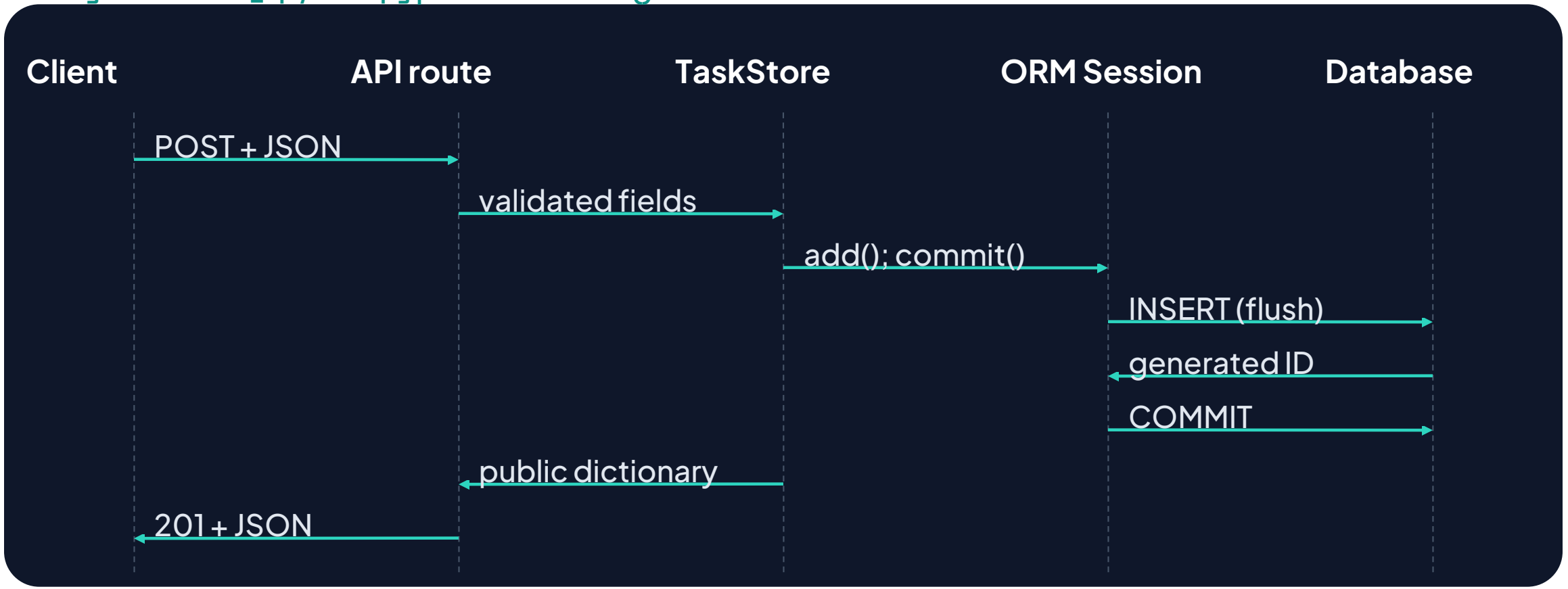
CHECK THE OBJECT STATE AND THE DATABASE STATE

Step	Effect
Task(...)	Construct a Python object
add(task)	Register it for insertion
flush()	Execute pending work inside the transaction
commit()	Flush remaining work and commit
rollback()	Discard the transaction's uncommitted changes

After flush, rollback can still undo the INSERT. An ID does not prove commit.

The Complete Process for Creating a Task

Existing route + task_api/store.py | TaskStore.create()



Only return 201 Created after commit succeeds.

Validation happens before TaskStore changes mapped data.

Apply It: Implement TaskStore.create()

task_api/store.py | method inside TaskStore

```
def create(self, fields):
    task = Task(
        title=fields["title"],
        done=fields.get("done", False)
    )
    db.session.add(task)
    self._commit()
    return task.to_dict()
```

Construct → add → commit → return the public representation.

The existing _commit() helper handles commit failures; we explain it shortly.

Reading Data: Build, Execute, Extract

THEORY | A STATEMENT IS NOT ITS RESULT

Stage	Meaning
db.select(Task)	Build a SQL expression; it has not run yet
db.session.execute(statement)	Run it and return a Result
result.scalars().all()	Collect the selected Task objects

get(Task, id) is a primary-key shortcut. A read does not need a write commit.

Apply It: Read One Task by Primary Key

task_api/store.py | method inside TaskStore

```
def get(self, task_id):
    task = db.session.get(Task, task_id)
    if task is None:
        return None
    return task.to_dict()
```

db.session.get(...) returns a Task or None; to_dict() builds the public result.

The route converts a missing result into 404 Not Found.

Apply It: Read and Serialize the Task List

task_api/store.py | method inside TaskStore

```
def list(self):
    statement = db.select(Task).order_by(Task.id)
    result = db.session.execute(statement)
    tasks = result.scalars().all()
    public_tasks = []
    for task in tasks:
        public_tasks.append(task.to_dict())
    return public_tasks
```

Build the query, execute it, collect objects, then create dictionaries.

order_by(Task.id) gives the list a predictable order.

How Does an Update Reach the Database?

Tracked object

ORM MECHANISM

Load the existing Task with get().

Assign a validated attribute value.

Flush can emit an UPDATE.

Partial update

PATCH SEMANTICS

Change only supplied fields.

Omitted fields keep their values.

False is a valid supplied value.

Validate the entire payload before mutation; commit the tracked changes.

Apply It: Update Only the Supplied Fields

task_api/store.py | method inside TaskStore

```
def update(self, task_id, fields):
    task = db.session.get(Task, task_id)
    if task is None:
        raise ApiError(404, "not_found", "Task not found")
    if "title" in fields:
        task.title = fields["title"]
    if "done" in fields:
        task.done = fields["done"]
    self._commit()
    return task.to_dict()
```

The presence test accepts False; commit flushes the tracked change.

A missing task produces 404. An omitted title remains unchanged.

Quick Check: Why Does Reopening a Task Fail?

Stored task and request

THE SITUATION

Stored value: task.done is True.

PATCH body: {"done": false}

Desired result: task.done is False.

Proposed Python test

FIND THE BUG

```
if fields.get("done"):
    task.done = fields["done"]
```

Does the assignment run? Why?

Discuss for 45 seconds: a missing value and a supplied False are different.

Apply It: Check Presence, Not Truthiness

task_api/store.py | excerpt inside TaskStore.update()

```
# The complete payload has already been validated.
if "done" in fields:
    task.done = fields["done"]

self._commit()
```

For {"done": false}, the assignment runs and stores False.

If "done" is omitted, the current value is preserved.

What Should Happen When a Write Fails?

THEORY | TRANSACTIONS AND RECOVERY

- **A transaction groups** changes that must succeed or fail together.
- **Validate the complete** request before changing mapped objects.
- **If flush or commit fails,** roll back before reusing that Session.
- **Return an error response.** A rollback cannot undo a previous successful commit.

A successful API write depends on a successful database transaction.

Next: see the starter's helper for failures during commit.

Apply It: Roll Back a Failed Commit

task_api/store.py | existing helper inside TaskStore

```
def _commit(self):
    try:
        db.session.commit()
    except SQLAlchemyError:
        db.session.rollback()
        raise ApiError(
            500, "database_error", "Database operation failed"
        )
```

Import SQLAlchemyError from sqlalchemy.exc; _commit() is our helper.

It handles errors in commit() and its implicit flush.

How Does Deleting a Task Work?

THEORY | FIND, MARK, FLUSH, COMMIT

- **Find the existing object** before trying to remove it.
- **Session.delete(object)** marks it for deletion; it is not a commit.
- **Flush sends DELETE inside** the transaction; commit completes it.
- **The store reports success** or absence. The route chooses 204 or 404.

Deletion follows the same transaction rules as creation and update.

Next: implement the store method for a task with no dependent rows.

Apply It: Delete an Existing Task

task_api/store.py | method inside TaskStore

```
def delete(self, task_id):
    task = db.session.get(Task, task_id)
    if task is None:
        return False
    db.session.delete(task)
    self._commit()
    return True
```

get → delete → commit. Return False when the task does not exist.

The route returns 204 on success and 404 for a missing task.

A New Requirement: Organize Tasks into Projects

One Project

projects

id: primary key

name: project name

Can contain many tasks.



Many Tasks

tasks

project_id: nullable foreign key

Each task has at most one project.

Tasks may remain unassigned.

projects.id ← tasks.project_id

Extension: migrate the schema or use a separate fresh demonstration database.

A Foreign Key and a Relationship Have Different Jobs

THEORY | STORED REFERENCE AND PYTHON NAVIGATION

Declaration	Meaning
ForeignKey("projects.id")	Database table and column
relationship("Project")	Python mapped class
back_populates="tasks"	Matching attribute on Project
Project.tasks ↔ Task.project	Paired Python relationships

Key = stored reference; relationship = object navigation.
SQLite: enable foreign-key checking on every connection.

Apply It: Define the Project Side

Extension | task_api/models.py | add the Project class

```
class Project(db.Model):
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    tasks = db.relationship(
        "Task",
        back_populates="project"
    )
```

id and name are columns. tasks is a collection of Task objects.

back_populates="project" names the attribute on Task.

Apply It: Define the Task Side

Extension | task_api/models.py | inside the existing Task class

```
project_id = db.Column(
    db.Integer,
    db.ForeignKey("projects.id"),
    nullable=True
)

project = db.relationship(
    "Project",
    back_populates="tasks"
)
```

project_id is stored in tasks; project navigates to a Project object.

Add these attributes inside the existing Task class.

Apply It: Save and Navigate Related Objects

Extension demonstration | inside an active app.app_context() block

```
project = Project(name="IN432 Coursework")
task = Task(title="Prepare project proposal", done=False)
task.project = project

db.session.add_all([project, task])
db.session.commit()

print(task.project.name)
for related_task in project.tasks:
    print(related_task.title)
```

One commit saves both objects; the ORM orders the dependent INSERTs.

Navigation: Task → one Project; Project → its Tasks.

How Do ORM Queries Combine Related Tables?

THEORY | JOINS AND RESULT SHAPES

- **An inner join keeps tasks** with a matching project.
- **A left outer join starting** from Task also keeps unassigned tasks.
- **Select a model for objects,** or selected columns for rows of values.
- **The relationship supplies** the join path. A join alone does not pre-load every relationship.

Decide which records to keep and which values to return before writing the query.

Next: read titles and project names, then include unassigned tasks.

Apply It: Read Titles and Project Names

Extension | task_api/store.py | inside an added query method

```
statement = (
    db.select(Task.title,
        Project.name.label("project_name"))
    .join(Task.project)
    .order_by(Task.id)
)
rows = db.session.execute(statement).all()
result = []
for row in rows:
    result.append({
        "title": row.title, "project": row.project_name
    })
```

Each result row has two selected values; serialize them explicitly.

Use all() here: scalars() would keep only the first selected element.

Apply It: Include Tasks Without a Project

Extension | task_api/store.py | inside an added query method

```
statement = (
    db.select(
        Task.title,
        Project.name.label("project_name")
    )
    .outerjoin(Task.project)
    .order_by(Task.id)
)
rows = db.session.execute(statement).all()
```

An unassigned task remains in the result with project_name = None.

Serialize None as JSON null when building the public dictionary.

Lab: Make the Existing Task API Persistent

Represent

TODO A

- Complete Task.to_dict().
- Expose id, title, and done.
- Return a plain dictionary.

Create

TODO B

- Construct and add a Task.
- Commit using the helper.
- Return its representation.

Update

TODO C

- Load the existing task.
- Apply supplied fields, including False.
- Commit the change.

Verify: create → restart the server → read the same task using its returned URL.

Check Your Understanding

PAIR DISCUSSION

1. Where does db.Model come from?
2. Does Task(...) insert a row?
3. How are flush() and commit() different?
4. Where does the task-list query belong?
5. Why check whether "done" is present?
6. How do project_id and project differ?
7. When do we need an outer join?

Discuss in pairs; explain each answer using an object and a database row.

Check the Explanations

CHECK YOUR EXPLANATIONS

1. db.Model is the base exposed by our Flask-SQLAlchemy object.
2. Task(...) constructs an object; it does not insert a row.
3. Flush sends pending work; commit completes the transaction.
4. Write the query in TaskStore.list(), in task_api/store.py.
5. Key presence accepts False as a valid requested value.
6. project_id stores a key; project gives access to an object.
7. Use an outer join to keep tasks with no matching project.

One request → validated fields → tracked objects → database transaction.

Official Documentation and Optional Extensions

CLICK A TITLE TO OPEN THE PRIMARY DOCUMENTATION

- Flask-SQLAlchemy Quickstart
- Application Context
- ORM Data Manipulation
- Relationship Patterns
- ORM Querying Guide
- Alembic Migrations

The next slides are optional reference: deeper relationship and query patterns.